SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO3: Articulation** (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	**Total Marks:**	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Y Greeshma Reddy	**Reg. No.:**	192424146
Section / Batch:	B Tech-AI&DS	**Date:**	03-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Q1.Evaluate Quick Sort performance in multi-core systems by distributing partitions across processors. Record execution time and compare with single-threaded execution. Analyze parallel speedup. Interpret results and justify scalability of Quick Sort in parallel environments.

Code of Conduct

I Y Greeshma Reddy (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ___Y Greeshma Reddy_____

RUBRICS FOR CALCULATION

Experiments					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				

Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Understanding of Concepts

- Explain Quick Sort algorithm
- Explain parallel processing / multi-core systems
- Define speedup and scalability
- Understanding of thread-based execution

2. Experimental Design

- Compare single-threaded vs multi-threaded execution
- Use different input sizes (small, medium, large)
- Distribute partitions properly among processors
- Maintain same testing conditions

3. Use of Tools

- Use programming languages (Python / C / C++)
- Apply threading / multiprocessing / OpenMP
- Use time measurement functions
- Implement parallel Quick Sort correctly

4. Analysis of Results

- Record execution time for both methods
- Calculate speedup = T_1 / T_p
- Use tables or graphs for comparison

- Identify overheads and performance issues
- Interpret efficiency and improvements

5. Documentation

- Include introduction, methodology, results, conclusion
- Present data clearly (tables/graphs)
- Explain observations properly
- Justify scalability of Quick Sort
- Write clear and structured conclusion

IMPLEMENTATION

Your Code

```python
import random
import time

def quicksort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[0]

    left = []
    right = []

    for x in arr[1:]:
        if x <= pivot:
            left.append(x)
        else:
            right.append(x)

    return quicksort(left) + [pivot] + quicksort(right)

n = int(input())

arr = [random.randint(1, 1000) for _ in range(n)]

start = time.time()

sorted_arr = quicksort(arr)

end = time.time()

print("Original Array:")
print(arr)

print("Sorted Array:")
print(sorted_arr)

print("Execution Time:", end - start, "seconds")
```

Submitted

Input

```
10
```

Output

```
Original Array:
[20, 286, 550, 522, 475, 708,
384, 835, 844, 588]
Sorted Array:
[20, 286, 384, 475, 522, 550,
588, 708, 835, 844]
```